

CAB432 Project Report

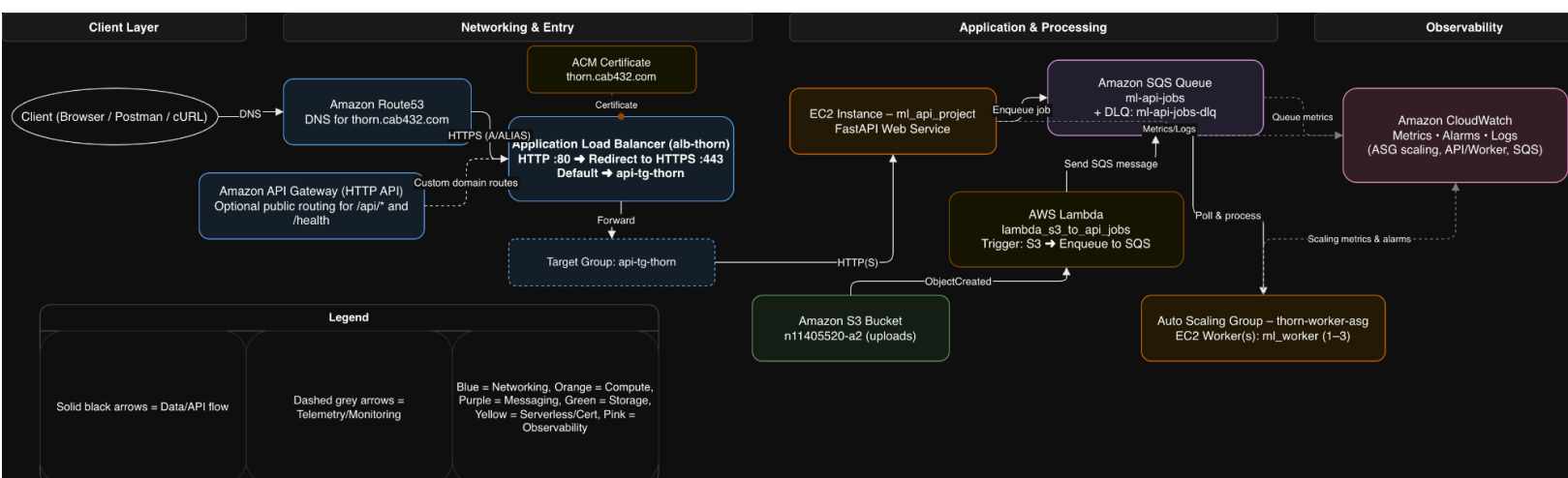
CAB432 – Cloud Computing

Author: Thorn Horwell – n11405520

Application Overview

The project implements a scalable, secure, and event-driven Machine Learning (ML) Anomaly Detection API on AWS. Users interact through a FastAPI-based web service deployed on EC2, where they can upload images for anomaly detection. The system queues these uploads for asynchronous processing by a CPU-intensive machine learning worker service. Tasks are distributed through Amazon SQS, handled by a worker cluster that scales automatically, with HTTPS access through a custom domain (<https://thorn.cab432.com>).

Application Architecture



The system is built using modular AWS services designed for scalability, reliability, and automation. Each service plays a distinct role:

- EC2 – Hosts the main FastAPI web service (`ml_api_project`) and the CPU-intensive worker (`ml_worker`) on separate instances.
- SQS – Queues processing jobs between API and worker services.
- Lambda – Triggered by S3 uploads to enqueue new jobs to SQS.

- S3 – Stores uploaded files (e.g., HAR data or images).
- ALB – Application Load Balancer distributing traffic and managing HTTPS termination.
- ACM – Provides SSL/TLS certificates for secure HTTPS communication.
- API Gateway – Routes requests externally to backend services.
- ASG – Auto Scaling Group for scaling worker instances between 1–3 based on CPU utilisation.
- CloudWatch – Collects metrics, alarms, and scaling data.
- SQS DLQ – Captures failed messages for review and troubleshooting.
- Route53 – Manages DNS for the thorn.cab432.com domain.

Justification of Architecture

The architecture was intentionally divided into microservices to enhance maintainability and scalability. The API service focuses on handling lightweight HTTP requests and job submission, while the worker service performs CPU-intensive tasks. This ensures the system remains responsive under load.

EC2 instances were chosen for both services to maintain control and visibility during development and testing. Lambda was used for lightweight, event-driven automation (S3 to SQS pipeline), simplifying orchestration without needing persistent infrastructure.

SQS was implemented to ensure reliable, asynchronous communication between microservices, providing durability and fault tolerance. The ALB and API Gateway handle external communication, ensuring secure HTTPS routing and load distribution.

The use of AWS-managed services like ACM, CloudWatch, and Route53 abstracts complexity, improves reliability, and supports continuous monitoring.

Response to Marking Criteria for Project

Project Core – Microservices

****Service 1 (API):**** FastAPI app (ml_api_project) – EC2 instance.

****Service 2 (Worker):**** CPU-bound task processor (ml_worker) – EC2 instance (Auto Scaling).

Project Core – Load distribution

****Mechanism:**** Application Load Balancer + SQS.

****ALB Name:**** thorn-alb.

****Target group:**** thorn-api-tg.

Project Core – Autoscaling

****ASG Name:**** thorn-worker-asg.

****Scaling policy:**** CPU utilisation target = 70%.

****Demonstration:**** Scaling 1 → 3 → 1 instances.

Project Core – HTTPS

****Domain:**** thorn.cab432.com.

****Certificate ID:**** 4e01fcbe47d8f5ceb82740b5aa48183101a3122bba5111e4fecdfd
1056f46cdc

****Load balancer:**** thorn-alb.

Project Additional – Serverless

****Lambda function:**** lambda_s3_to_api_jobs.

****Trigger:**** S3 upload → invokes Lambda → enqueues to SQS.

Project Additional – Dead Letter Queue

****DLQ Name:**** thorn-worker-dlq.

****Technology:**** SQS with redrive policy (maxReceiveCount = 5).

Project Additional – Communication mechanisms

****Mechanisms:**** SQS + API Gateway.

****Gateway ID:**** po03l5e37l.

Cost Estimate

AWS Pricing Calculator Link:

<https://calculator.aws/#/estimate?id=cd00e7abb4c2d8e744c545adb0b95439b47911bf>

Service	Estimated Monthly Cost (USD)
Amazon EC2 (API + Worker instances)	\$38.58

Elastic Load Balancing (ALB)	\$18.81
Amazon SQS (Standard + DLQ)	\$2.30
AWS Lambda	\$0.00
Amazon S3 (Storage + Requests)	\$0.51
Amazon CloudWatch (Metrics + Logs)	\$6.75
Amazon Route 53 (DNS)	\$0.90
Total Estimated Monthly Cost	≈ \$67.85 USD (~ \$105 AUD)

Scaling Up

To scale up for 10,000 concurrent users, the API and worker would migrate to ECS or AWS Fargate for containerised orchestration, providing better horizontal scaling. CloudFront could be introduced for content caching, while SQS batching would increase worker throughput. A database such as DynamoDB could handle user/job persistence.

Security

- **IAM Roles:** Least privilege – EC2, Lambda, and ASG use scoped IAM roles.
- **S3 Bucket Policy:** Private access only with signed URLs.
- **HTTPS Enforcement:** TLS via ACM and ALB ensures encryption in transit.
- **Security Groups:** Restrict inbound ports, allowing only HTTP/HTTPS.
- **DLQ Logging:** Captures failed messages for accountability.
- **CloudWatch Alerts:** Monitor scaling, errors, and unauthorised access attempts.

Sustainability

The application was designed with efficiency in mind at every level:

- **Software:** Dockerised microservices reduce resource overhead.
- **Hardware:** t2.micro instances minimise idle compute usage.
- **Data Centre:** Hosted in AWS Sydney region powered by renewable energy.
- **Resources:** Autoscaling optimises compute utilisation and reduces waste.